Module 06 Lab - Regression and Classification Models

Objective: To understand the difference between regression and classification and to build your first linear models for both tasks.

In this lab, you will write the code to train and evaluate the models.

Group Members

Group: FA26_ML_6171_16347 5

Person 1: Kristina Akpan (Part 1, Regression vs. Classification, Knowledge Check 1, notebook assembly)

Person 2: Angel Martinez (Part 2, Linear Regression code)

Person 3: Kerry Walker (Linear Regression interpretation, Knowledge Check 2 and 3)

Person 4: Thomas Jaymon (Part 3, Logistic Regression code)

Person 5: Jamie Lopez (Logistic Regression interpretation and reflection)

Part 1: Regression vs. Classification

This is the most fundamental distinction between types of supervised learning problems.

- **Regression:** The goal is to predict a **continuous numerical value**.
    - *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.
- **Classification:** The goal is to predict a **discrete category or class label**.
    - *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.

In this lab, we will tackle one of each.

Completed by: Kristina Akpan (Person 1)

```
# --- Part 1: Looking at a Regression Target vs. a Classification Target ---
# Completed by: Kristina Akpan (Person 1)

import pandas as pd

# Load the Titanic dataset (same file used in Parts 2 and 3)
df_look = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
print("--- Dataset Shape (rows, columns) ---")
print(df_look.shape)
print()

# Fare: the regression target. It is a continuous number with many possible values.
print("--- Fare (regression target) ---")
print(f"Number of different fare values: {df_look['Fare'].nunique()}")
print(f"Lowest fare: {df_look['Fare'].min():.2f}   Highest fare: {df_look['Fare'].max():.2f}   Average fare: {df_look['Fare'].mean():.2f}")
print(df_look['Fare'].head())
print()

# Survived: the classification target. It only has two possible labels, 0 or 1.
print("--- Survived (classification target) ---")
print(f"Number of different values: {df_look['Survived'].nunique()}")
print(df_look['Survived'].value_counts())
print()

# Quick check for a regression vs. classification problem:
# a numeric target with many unique values is regression, a small set of labels is classification
for col in ['Fare', 'Survived']:
    kind = "Regression" if df_look[col].nunique() > 10 else "Classification"
    print(f"{col}: {df_look[col].nunique()} unique values, so it is a {kind} problem")
```

```
--- Dataset Shape (rows, columns) ---
(891, 12)

--- Fare (regression target) ---
Number of different fare values: 248
Lowest fare: 0.00   Highest fare: 512.33   Average fare: 32.20
0     7.2500
1    71.2833
2     7.9250
3    53.1000
4     8.0500
Name: Fare, dtype: float64

--- Survived (classification target) ---
Number of different values: 2
Survived
0    549
1    342
Name: count, dtype: int64
```

```
Fare: 248 unique values, so it is a Regression problem
Survived: 2 unique values, so it is a Classification problem
```

Part 1 observations: Kristina Akpan (Person 1)

The dataset has 891 passengers (rows) and 12 columns.

Fare has 248 different values, ranging from 0.00 to 512.33 with an average of about 32.20. Because it can be almost any amount, predicting Fare is a regression problem.

Survived only has 2 values, 0 and 1. 549 passengers did not survive (about 61.6%) and 342 survived (about 38.4%). Because the answer is one of two labels, predicting Survived is a classification problem.

This is the same dataset used in Parts 2 and 3. The only thing that changes the type of problem is the column we choose to predict.

Part 2: Linear Regression

Concept: Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".

- **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

Completed by: Angel Martinez (Person 2)

```
# Completed by: Angel Martinez (Person 2)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)

# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
```

Task 1: Train and Evaluate a Linear Regression Model

Your Task:

1. Create an instance of the `LinearRegression` model.
2. Train the model using the training data (`X_train_reg`, `y_train_reg`).
3. Make predictions on the test data.
4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

Completed by: Angel Martinez (Person 2)

```
# Completed by: Angel Martinez (Person 2)
# --- ENTER YOUR CODE HERE ---
import numpy as np  # needed for np.sqrt below

# 1. Create the model instance
# lr_model = ...
lr_model = LinearRegression()  # an untrained linear regression model

# 2. Train the model
# lr_model.fit(...)
lr_model.fit(X_train_reg, y_train_reg)  # learns one slope per feature plus an intercept

# 3. Make predictions
# y_pred_reg = lr_model.predict(...)
y_pred_reg = lr_model.predict(X_test_reg)  # predicted fares for passengers the model has not seen

# 4. Evaluate the model
# mse = mean_squared_error(y_test_reg, y_pred_reg)
mse = mean_squared_error(y_test_reg, y_pred_reg)  # average squared gap between actual and predicted fare
# print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
# print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is off by about $48 on average.")
print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is off by about ${np.sqrt(mse):.0f} on average.")

# Coefficients for the Knowledge Check (one value per feature, in the order of features)
print("Features:", features)
```

```
print("lr_model.coef_:", lr_model.coef_)
print(f"lr_model.intercept_: {lr_model.intercept_:.2f}")
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01, meaning our model is off by about $58 on average.
Features: ['Age', 'Pclass']
lr_model.coef_: [ -0.43695889 -38.29728935]
lr_model.intercept_: 133.31
```

Interpreting the Linear Regression Results

Completed by: Kerry Walker (Person 3)

The model gave a mean squared error of 3364.92. Because MSE is in squared dollars, it is easier to read its square root, about 58.01, which means a typical prediction misses the real fare by about 58 dollars. (The comment in the starter code mentions about 48 dollars, but the actual run on this split gives about 58.) To put that in context, the average fare in the test set is about 33 dollars and the median is about 14 dollars, so a 58 dollar typical error is large. The R squared score on the test set is only about 0.19, so Age and Pclass explain a small part of why fares vary.

The coefficients explain what the model learned. Pclass has a coefficient of about negative 38.30, so each step down in class lowers the predicted fare by roughly 38 dollars. Age has a coefficient of about negative 0.44, so age barely changes the prediction. Part of the reason the error is so high is that a few first class tickets cost over 500 dollars, and a straight line cannot fit those extreme values well. Squaring the errors makes those big misses weigh heavily in the MSE. The model captures the general pattern that higher classes paid more, but it is not precise enough to predict an individual passenger's fare.

Part 3: Logistic Regression

Concept: Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.

- **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

Completed by: Thomas Jaymon (Person 4)

```
# Completed by: Thomas Jaymon (Person 4)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Prepare data for classification
# We need to encode 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'

X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls, test_size=0.2, random_state=42)
```

Task 2: Train and Evaluate a Logistic Regression Model

Your Task:

1. Create an instance of the `LogisticRegression` model.
2. Train the model using the classification training data.
3. Make predictions on the test data.
4. Evaluate the model using `accuracy_score`.

Completed by: Thomas Jaymon (Person 4)

```
# Completed by: Thomas Jaymon (Person 4)
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
# log_model = ...
log_model = LogisticRegression()  # an untrained logistic regression classifier

# 2. Train the model
# log_model.fit(...)
log_model.fit(X_train_cls, y_train_cls)  # learns how Age, Pclass and Sex relate to survival

# 3. Make predictions
# y_pred_cls = log_model.predict(...)
y_pred_cls = log_model.predict(X_test_cls)  # predicted class (0 or 1) for each test passenger

# 4. Evaluate the model
# accuracy = accuracy_score(y_test_cls, y_pred_cls)
accuracy = accuracy_score(y_test_cls, y_pred_cls)  # share of test passengers predicted correctly
# print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")

# Extra output so the results can be interpreted
from sklearn.metrics import confusion_matrix
print(f"Test passengers: {len(y_test_cls)}")
print(f"Baseline (always predict did not survive): {(y_test_cls == 0).mean():.2%}")
print("Confusion matrix (rows = actual 0/1, columns = predicted 0/1):")
print(confusion_matrix(y_test_cls, y_pred_cls))
print("Features:", features_cls)
print("log_model.coef_:", log_model.coef_)
```

```
Accuracy for Survival Prediction: 74.83%
Test passengers: 143
Baseline (always predict did not survive): 60.84%
Confusion matrix (rows = actual 0/1, columns = predicted 0/1):
[[68 19]
 [17 39]]
Features: ['Age', 'Pclass', 'Sex']
log_model.coef_: [[-0.04253228 -1.24873462  2.53219712]]
```

Interpreting the Logistic Regression Results

Completed by: Jamie Lopez (Person 5)

The model reached an accuracy of 74.83% on the 143 test passengers, which means it predicted survival correctly for 107 of them and incorrectly for 36. A useful comparison is a baseline that always guesses "did not survive." Since about 60.84% of the test passengers did not survive, that lazy guess would already be right 60.84% of the time. Our model beats it by about 14 percentage points, so Age, Pclass and Sex carry real information about who survived.

The confusion matrix shows where the mistakes happen. Of the 87 passengers who did not survive, the model got 68 right and wrongly predicted survival for 19. Of the 56 who survived, it got 39 right and missed 17. So the model is better at recognizing passengers who died than those who survived.

The coefficients agree with the history of the Titanic. Sex has the largest positive coefficient, about 2.53, so being female strongly raised the predicted chance of survival. Pclass has a negative coefficient, about negative 1.25, so lower classes had lower survival chances. Age is slightly negative, about negative 0.04 per year, so older passengers were a little less likely to survive. Overall the model performs reasonably well for only three simple features, but about one in four predictions is still wrong, so it should be seen as a starting point rather than a finished model.

📝 Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **In your own words, what is the key difference between a regression problem and a classification problem?**
2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**
3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model?** Why wouldn't accuracy be a good metric for the fare prediction task?

Answers

1. The key difference between a regression problem and a classification problem

Completed by: Kristina Akpan (Person 1)

The difference is the kind of answer the model gives back. A regression problem predicts a number that can fall anywhere on a scale, like a ticket fare of 7.25 or 71.28 dollars, a house price, or tomorrow's temperature. A classification problem predicts which group something belongs to from a fixed set of options, like survived or did not survive, spam or not spam, or which of three flower species a sample is. A simple way to tell them apart is to ask whether "how much" or "which one" is the question. In this lab, predicting Fare is regression because fare is a continuous amount, and predicting Survived is classification because the only possible answers are 0 and 1.

2. What the numbers in lr_model.coef_ represent

Completed by: Kerry Walker (Person 3)

When printed, lr_model.coef_ gives two numbers, about negative 0.44 and negative 38.30, one for each feature in the order they were listed, Age then Pclass. Each coefficient is the slope for that feature. It tells us how much the predicted fare changes when that feature goes up by one while the other feature stays the same. The Age coefficient of about negative 0.44 means each extra year of age lowers the predicted fare by about 44 cents, which is a very small effect. The Pclass coefficient of about negative 38.30 means moving down one class (from 1st to 2nd, or 2nd to 3rd) lowers the predicted fare by about 38 dollars. The sign shows the direction and the size shows the strength, so passenger class matters far more for fare than age does. The intercept, about 133.31, is the starting value the model adds these effects to.

3. Why mean_squared_error for regression but accuracy_score for classification

Completed by: Kerry Walker (Person 3)

The metric has to match the kind of answer. Mean squared error measures how far off each numeric prediction is, squares those gaps so bigger misses count more, and averages them. That makes sense for fare, where being off by 2 dollars is much better than being off by 200 dollars. Our MSE was 3364.92, and its square root is about 58, so our fare predictions are off by roughly 58 dollars on average. Accuracy simply counts the share of predictions that are exactly right, which works for survival because each prediction is either the correct class or the wrong one.

Accuracy would not be a good metric for fare prediction because a predicted fare will almost never match the real fare exactly to the cent. A prediction of 30.10 for a ticket that cost 30.00 would be counted as wrong, the same as a prediction of 300.00. Accuracy would come out near zero and would not tell us whether the model is close or far away, while MSE rewards predictions that land near the real value.

Reflection

Completed by: Jamie Lopez (Person 5)

What we learned: This lab made the difference between regression and classification concrete. We used the same Titanic data for both tasks, and the only thing that changed the type of problem was the target column. Predicting Fare gave us numbers on a scale, while predicting Survived gave us a yes or no answer. We also learned that the scikit learn workflow is the same for both models: create the model, fit it on training data, predict on test data, and score it. What changes is how we judge the result.

Challenges: The starter code used np.sqrt without importing NumPy, so we had to add the import before the cell would run. The comment also said the model would be off by about 48 dollars, but our run gave about 58, so we printed the real value instead of copying the comment. Reading MSE was confusing at first because 3364.92 looks huge until you take the square root and turn it back into dollars.

Insights: A single accuracy number can look better than it really is. Comparing 74.83% to the 60.84% baseline showed how much the model actually learned, and the confusion matrix showed that most of its mistakes were missed survivors. On the regression side, a few very expensive tickets pulled the error up, which showed us how sensitive MSE is to outliers.

Connections to previous labs: In our earlier NLP labs we turned text into features, such as word counts and part of speech tags. This lab showed what happens after that step. Models like logistic regression are what take those features and make predictions, for example sorting reviews as positive or negative or emails as spam or not spam. Encoding Sex as 0 and 1 here is the same idea as turning words into numbers before a model can use them.